

El conjunto que dice no

Matroides transversales en Lean 4: una prueba del axioma de aumento por conjuntos *tight*, y la obstrucción canónica que hace posible

Carles Marín*

Investigador independiente

karlesmarin@gmail.com

Julio de 2026

Resumen

Mathlib tiene el teorema de los matrimonios de Hall. Mathlib tiene matroides. En la revisión contra la que se ha verificado este trabajo (84fdd0bf, 15-07-2026) nada une ambas cosas: todo matroide de `Matroid/Constructions.lean` es trivial (`emptyOn`, `loopyOn`, `freeOn`, `uniqueBaseOn`), y ninguno se construye a partir de datos combinatorios. Este artículo cierra ese hueco para el puente clásico entre ambas, el *teorema del matroide transversal* de Edmonds–Fulkerson y, de forma independiente, de Mirsky–Perfect: para una familia finita $A = (A_0, \dots, A_{t-1})$ de conjuntos finitos, los transversales parciales de A son exactamente los conjuntos independientes de un matroide. Presento una formalización completa y sin `sorry` en Lean 4 sobre Mathlib de la forma de presentación finita, empaquetada como `transversalMatroid A`, con `transversalMatroid_indep` identificando sus conjuntos finitos independientes.

La contribución no es que Lean acepte la prueba, sino la *forma* de la prueba. En lugar de certificar un algoritmo de caminos de aumento, el desarrollo aplica el método de obstrucción de Hall y aísla un objeto en una capa propia: un conjunto *tight* R , uno cuyas ranuras compatibles quedan exactamente saturadas, $|N(R)| = |R|$. Una inserción fallida produce un testigo *tight*; los conjuntos *tight* son cerrados por unión; y una unión de testigos fuerza $|J \setminus I| \leq |I \setminus J|$, en contradicción con $|I| < |J|$. Cada paso es un lema independiente y con nombre propio sobre la interfaz de emparejamientos finitos de Mathlib, y esa separación paga un dividendo que el enunciado clásico no anuncia. Como los conjuntos *tight* son cerrados también por intersección, forman un *retículo*, de modo que todo transversal parcial I lleva consigo un único mayor subconjunto *tight* `maxTight A I`, y

$$\text{insert } e \text{ I es un transversal parcial} \iff N(e) \not\subseteq N(\text{maxTight } A \text{ I}),$$

un solo conjunto canónico decide toda inserción (`insert_iff_not_subset_maxTight`). Dicho en el vocabulario propio de Mathlib, ese mismo conjunto calcula el operador de clausura del matroide sobre conjuntos independientes (`mem_closure_iff_subset_maxTight`): el aparato de la biblioteca aplicándose al objeto construido, y no solo declarado aplicable. Seiscientos cuarenta y cinco líneas de Lean, treinta y cuatro teoremas, sin `sorry`, `admit` ni `native_decide`; `#print axioms` sobre los seis teoremas principales devuelve únicamente `propxt`, `Classical.choice` y `Quot.sound`. La Sección 2 sitúa el trabajo frente a las formalizaciones de Hall, de emparejamientos y de matroides en Lean, Isabelle/HOL, Mizar y Coq; y la Sección 10 declara exactamente qué afirmaciones de prioridad se hacen, qué se buscó para sostenerlas, en qué fecha, y qué las refutaría.

*Formalizado con asistencia de un sistema de IA (Claude, Anthropic); las matemáticas son clásicas, y toda afirmación, delimitación de alcance y limitación es responsabilidad del autor. Lo que *no* está formalizado se expone sin adornos en la Sección 9, y quien certifica las pruebas es el núcleo de Lean, no el asistente.

Lo que hay aquí. El artículo sigue el desarrollo en el orden en que se construyó (Figura 1). Las Secciones 1–2 dicen dónde se detiene el estado del arte formal y qué se añade exactamente. La Sección 3 sustituye la búsqueda de un emparejamiento por una desigualdad de conteo; la Sección 4 convierte el *fallo* de esa desigualdad en un objeto y prueba el aumento a partir de él; y la Sección 5 explica por qué ese camino, partido en tres capas, es la contribución y no una preferencia de estilo. La Sección 6 empaqueta el matroide; la Sección 7 recoge el dividendo que paga esa arquitectura. Los lemas que sostienen el edificio están en la Sección 8, los límites honestos en la Sección 9 y la conclusión en la Sección 11. Si solo se van a leer dos cosas, léase el recuadro de contribución al final de la Sección 1 y el Teorema 1.

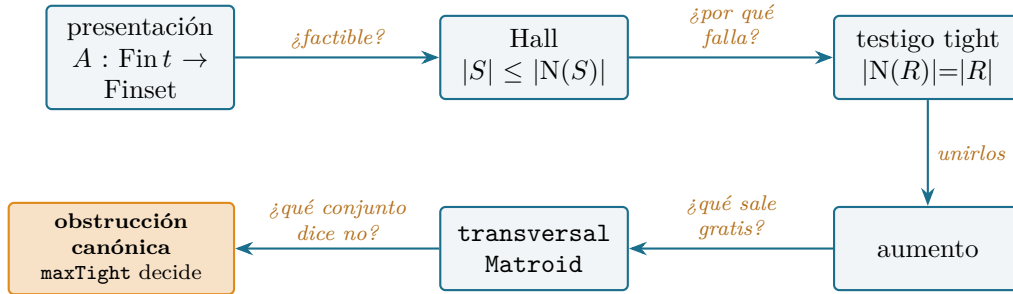


Figura 1: El mapa del lector. Cada nodo es un bloque de Lean sin **sorry** y cada flecha es la pregunta que responde su sección. La cadena va de izquierda a derecha y vuelve: de una presentación finita a la desigualdad de Hall; del *fallo* de esa desigualdad a un conjunto tight; de muchos conjuntos tight a uno solo; y del matroide al único conjunto canónico que decide toda inserción.

1. Dos bibliotecas que no se encuentran

Hay que asignar tareas a agentes cualificados distintos. Una tarea puede admitir varios agentes compatibles, dos tareas no pueden compartir agente, y la pregunta es qué colecciones de tareas pueden atenderse *a la vez*. La relación de compatibilidad es local e inofensiva; la condición de factibilidad no lo es, porque admitir una tarea nueva puede obligar a reasignar varias antiguas. Este es el terreno de los sistemas de representantes distintos y del teorema de los matrimonios de Hall [1].

Escribamos la familia presentadora como

$$A = (A_0, \dots, A_{t-1}), \quad A_i \subseteq \alpha,$$

llamemos a x *compatible* con el índice i cuando $x \in A_i$, y llamemos *transversal parcial* a un conjunto finito $I \subseteq \alpha$ cuando sus elementos pueden asignarse inyectivamente a índices compatibles. El conjunto vacío es factible; un subconjunto de un conjunto factible es factible, sin más que restringir la asignación. Ninguna de las dos observaciones alcanza el hecho que importa:

si I y J son transversales parciales y $|I| < |J|$, entonces algún $e \in J \setminus I$ hace de $I \cup \{e\}$ un transversal parcial.

Ese es el axioma de intercambio de los matroides, y no se ve en la definición. Su historia merece un párrafo, porque este artículo la repite.

Dos artículos de 1935. En el *Journal of the London Mathematical Society*, Philip Hall dio la condición exacta para que una familia de conjuntos admita un sistema de representantes distintos [1]. En el *American Journal of Mathematics*, Hassler Whitney abstraigo las propiedades de la dependencia

lineal y llamó *matroide* al resultado [10]. Compartieron el año y poco más: uno preguntaba cuándo puede hacerse un conjunto de elecciones, el otro axiomatizaba qué significa que unas elecciones sean independientes, y ninguno remite al otro.

Siguieron separados durante una generación. El artículo de Whitney despertó un interés breve y después la teoría de matroides quedó en silencio más de veinte años, hasta que los artículos de Tutte de 1958–59 caracterizaron los matroides regulares y gráficos [11]; entretanto Rado había seguido las mismas ideas en los años cuarenta, y probó la generalización matroidal del criterio de Hall [6]. El renacimiento llegó en los sesenta, y con él dos lugares donde por fin se leyeron juntos los dos artículos de 1935. Jack Edmonds convocó el primer congreso de teoría de matroides en el National Bureau of Standards en 1965 [9]; y en Sheffield, Leon Mirsky, Hazel Perfect y John Pym levantaban la teoría transversal a partir del teorema de Hall y sus generalizaciones. Ambos grupos llegaron al mismo puente. El *Journal of Research of the NBS* publicó la versión de Edmonds y Fulkerson el año del congreso [2]; la de Mirsky y Perfect, que además cubre familias infinitas, apareció en 1967 [3]. Treinta años después de 1935, el teorema de los matrimonios y los axiomas de la independencia resultaron tratar de un mismo objeto.

Perfect siguió por ese camino. En 1968 aplicó el teorema de Menger para producir la clase de matroides que hoy llamamos *gammoides* [4]—una clase que contiene a los matroides transversales [17, 18], de modo que el objeto formalizado en este artículo es un caso particular de uno que ella introdujo. Su doctorado llegó al año siguiente, en 1969, con el título *Studies in Transversal Theory with Particular Reference to Independence Structures and Graphs* [7]: la tesis vino después del teorema. Fue *Reader* en Sheffield, escribió *Independence Theory in Combinatorics* con Victor Bryant en 1980 [12], y la conjetura que formuló con Mirsky en 1965 sobre qué puntos del disco unidad son autovalores de matrices doblemente estocásticas $n \times n$ [8] —probada por ellos para $n \leq 3$, confirmada para $n = 4$ y refutada para $n = 5$ — sigue abierta más allá. Es su nombre el que pone este resultado en la lista de teoremas de mujeres matemáticas de *Theorem of the Day* [5], de donde se tomó el objetivo; la atribución que se usa en todo el artículo es la que imprime esa página.

Por qué importa en la práctica es el corolario voraz. Decir que los transversales parciales forman un matroide es decir exactamente que un transversal maximal más barato puede elegirse de forma voraz—primero los agentes más baratos, la asignación después—, lo cual es falso para la búsqueda ingenua de emparejamientos, donde un prefijo voraz puede haber que abandonarlo [5]. La página de *Theorem of the Day* termina con una reserva que este artículo se toma en serio y responde en la Sección 7: la voracidad “deja abierta la cuestión de cómo sabemos que nuestros transversales parciales son válidos si no encontramos un emparejamiento al mismo tiempo”.

Dónde está el estado del arte formal—es decir, en 1935. La situación que aborda este artículo es la anterior a 1965, reproducida dentro de un asistente de pruebas. Mathlib [26], la biblioteca matemática de Lean 4 [27], tiene el teorema de los matrimonios de Hall, aportado por Gusakov, Mehta y Miller [19], en la forma fuerte que necesitamos: una interfaz de emparejamientos finitos (`hallMatchingsOn`) y la equivalencia `Finset.all_card_le_biUnion_card_iff_existsInjective`.

Mathlib tiene además un desarrollo sustancial de matroides, debido en gran medida a Peter Nelson, con bases, circuitos, clausura, dualidad, menores y rango. Lo que no tiene es un puente. Todo matroide de Mathlib/Combinatorics/Matroid/Constructions.lean es trivial—`emptyOn`, `loopyOn`, `freeOn`, `uniqueBaseOn`—y ninguno se construye a partir de datos combinatorios. Buscar *transversal* en la revisión 84fdd0bf devuelve el sentido de teoría de grupos (clases laterales, Schreier, Schur–Zassenhaus) y una mención en prosa dentro del módulo de Hall; no hay matroide transversal. Las dos bibliotecas están una al lado de la otra y no se encuentran.

Qué aporta este artículo

Las formalizaciones existentes de emparejamientos y del teorema de Hall no dan por sí solas un matroide transversal: certifican *que* existe un emparejamiento, y el axioma de matroide necesita una razón de *por qué* falla una inserción. La contribución de este artículo es triple.

1. **La construcción.** `transversalMatroid A`, con sus conjuntos finitos independientes identificados exactamente (`transversalMatroid_indep`), enchufada a la API de matroides de Mathlib a través de `IndepMatroid.ofFinset`. Hasta donde sé, y frente a las búsquedas fechadas y detalladas en la Sección 10, es el primer matroide de Mathlib construido a partir de una presentación combinatoria y no por una construcción trivial o dual—una afirmación sobre lo que busqué, no una prueba de ausencia.
2. **La arquitectura de la prueba, que es lo esencial.** El aumento se prueba mediante un argumento de obstrucción de Hall organizado en torno a conjuntos *tight*, repartido en tres capas independientes—emparejamiento, conjuntos tight, conteo—, cada una un lema con nombre propio. No se formaliza ningún algoritmo de caminos de aumento, y no aparece ningún bloque monolítico de tácticas. La capa de conjuntos tight tiene forma de biblioteca: habla de familias de conjuntos y de la desigualdad de Hall, no de matroides.
3. **El dividendo que paga esa arquitectura.** Como la obstrucción quedó aislada, puede uno preguntarse qué hacen los conjuntos tight *como familia*. Son cerrados por intersección además de por unión, luego forman un retículo; por tanto todo transversal parcial tiene un subconjunto tight *máximo*, `maxTight A I`, y una sola prueba de contención contra él decide toda inserción (Teorema 1). Eso no se ve en el enunciado clásico del teorema, y es lo que la prueba por capas vuelve barato.

2. Trabajo relacionado

El objeto de esta sección es situar, no reclamar prioridad. Tres literaturas convergen en el objetivo de este trabajo—los teoremas de Hall formalizados, la teoría de emparejamientos formalizada y la teoría de matroides formalizada—, y el matroide transversal queda justo donde ninguna de las tres termina.

El teorema de los matrimonios de Hall, formalizado. El criterio de Hall de 1935 [1] es un objetivo habitual de los asistentes de pruebas. En Isabelle/HOL, Jiang y Nipkow aportaron *Hall's Marriage Theorem* al Archive of Formal Proofs en 2010 [20], con la versión finita y la de índice numerable. En Lean lo formalizaron Guskov, Mehta y Miller dentro de Mathlib [19], en tres formulaciones intercambiables (familias indexadas de `Finset`, una relación sobre tipos finitos y una forma con filtro sobre `Fintype`), junto con el paso de compacidad de Halpern que eleva el teorema finito a tipos de índice arbitrarios. Teoremas vecinos sobre conjuntos finitos se han mecanizado en otros sitios—los teoremas de Dilworth y de Mirsky en Coq [22], y de nuevo el de Dilworth en el AFP (2025)—, de modo que el teorema de los matrimonios está en la parte asentada del paisaje de la combinatoria formal, no en su frontera. *Qué da esto y qué no.* Estos desarrollos certifican la existencia de un sistema de representantes distintos bajo las desigualdades de Hall. No dicen qué ocurre cuando las desigualdades fallan, y el axioma de matroide es enteramente una afirmación sobre el fallo.

La teoría de emparejamientos, formalizada. Más allá del teorema de existencia, la teoría de emparejamientos formal se ha concentrado en los algoritmos y en los teoremas de estructura más profundos: el criterio de caminos de aumento de Berge, el emparejamiento máximo por *blossoms* y el rincón algorítmico de la optimización combinatoria, sobre todo en Isabelle/HOL. La entrada más directamente relevante es el desarrollo de ITP 2025 *A Formal Analysis of Algorithms for Matroids and Greedoids* [23], que incluye emparejamiento bipartito máximo ejecutable y algoritmos voraces verificados. Es el trabajo previo más cercano a este objetivo, y la razón de que aquí no se haga ninguna afirmación algorítmica: esa literatura ya es dueña del lado algorítmico. *Qué da esto y qué no.* Una rutina ejecutable de emparejamiento máximo decide la factibilidad, y puede recuperarse una obstrucción inspeccionando una búsqueda fallida. Pero entonces el certificado es un subproducto de una ejecución concreta del algoritmo, no un objeto con teoría propia; nada en ese camino te dice que las obstrucciones formen un retículo.

Los matroides, formalizados. Isabelle/HOL tiene la entrada del AFP de Keinholtz *Matroids* (2018) [21], que desarrolla conjuntos independientes, bases, circuitos y rango, además del trabajo de ITP 2025 sobre algoritmos de matroides y greedoides ya citado. En Lean 4 el mayor desarrollo de matroides es el de Peter Nelson, que sigue los axiomas de matroides infinitos y cuyo núcleo se ha incorporado a Mathlib; trabajos recientes del mismo ecosistema incluyen una formalización de la caracterización por menores excluidos de los matroides binarios [24] y una verificación de la dirección de composición del teorema de Seymour para matroides regulares [25]. El área de matroides en Lean está viva y es estructural. *Qué da esto y qué no.* Estas bibliotecas son ricas en lo que uno puede probar *acerca de* un matroide y pobres en matroides sobre los que probarlo. En concreto, en la revisión 84fdd0bf (15-07-2026) el módulo de construcciones ofrece únicamente los matroides vacío, de lazos, libre y de base única. Los matroides gráficos, los lineales y los transversales—las tres familias clásicas que hacen de la teoría de matroides una teoría de ejemplos—están, en esa revisión, ausentes.

La teoría transversal, formalizada. En búsquedas realizadas en julio de 2026 y detalladas en la Sección 10, no localicé ninguna formalización del teorema del matroide transversal: ni en Lean/Mathlib (una búsqueda de texto en la revisión 84fdd0bf devuelve solo el sentido de teoría de grupos de *transversal*), ni en el AFP de Isabelle, ni en Mizar, ni en el ecosistema *mathcomp* de Coq, ni en HOL Light. Esto es un resultado negativo de búsqueda y así se consigna en la Sección 9: es un estado de auditoría, no una reclamación de novedad, y el trabajo vecino del AFP sobre algoritmos de matroides y greedoides está lo bastante cerca como para que cualquier afirmación de primera formalización del *área* circundante fuese falsa.

El hueco, dicho con exactitud. Los teoremas de Hall formales certifican factibilidad; el trabajo formal sobre emparejamientos certifica algoritmos; el trabajo formal sobre matroides certifica estructura sobre un almacén vacío de ejemplos. El matroide transversal necesita las tres cosas a la vez, y la unión no es mecánica: el axioma de matroide es una afirmación sobre el fallo *simultáneo* de muchas inserciones, y ningún teorema de existencia, por fuerte que sea, la entrega. Proveer esa unión—con un objeto-obstrucción factorizado, de manera que pueda reutilizarse y hacersele preguntas—es lo que hace este artículo.

3. La factibilidad como desigualdad de conteo

Para un elemento x , el desarrollo en Lean nombra sus índices compatibles `compatibleSlots A x : Finset (Fin t)`, y para un S finito escribe

$$N(S) = \bigcup_{x \in S} \text{compatibleSlots } A x \quad (\text{slotsOf } A \ S).$$

`IsPartialTransversal A I` se define como la no vacuidad de `hallMatchingsOn (compatibleSlots A) I` de Mathlib: una función inyectiva sobre el subtipo de los elementos de I , con imagen en las ranuras compatibles. Poner el objeto en el lenguaje propio de la biblioteca, en vez de en una definición a medida, es lo que hace corto el resto del desarrollo.

El primer resultado sustantivo es la equivalencia finita exacta

$$I \text{ es un transversal parcial} \iff |S| \leq |N(S)| \text{ para todo } S \subseteq I.$$

La dirección directa restringe un emparejamiento a S e inyecta S en $N(S)$; la recíproca delega la construcción de representantes en el teorema de Hall finito de Mathlib sobre el subtipo de los elementos de I . La asimetría es ingeniería de pruebas deliberada: una dirección es una inyección de cardinales que podemos hacer a mano, la otra es exactamente el trabajo de la biblioteca. En Lean las dos mitades son `partialTransversal_hall` y `hall_partialTransversal`, emparejadas como `partialTransversal_iff_hall`.

- tight block $R = \{a, b, c\}$ and its three slots
- d : slack, outside the obstruction
- x refused: $N(x)$ lies inside $N(R)$
- y admitted: $N(y)$ escapes $N(R)$

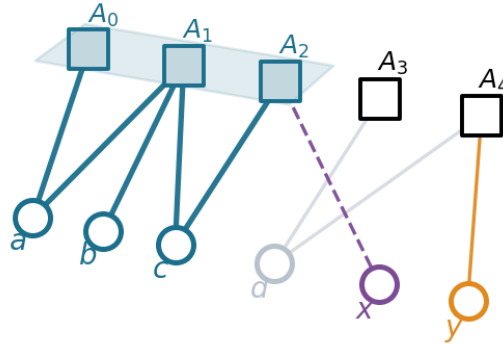


Figura 2: El ejemplo que recorre el artículo, dibujado por `generate_obstruction_figures.py` a partir de los mismos datos que verifica `check_canonical_obstruction.sage`. Cinco conjuntos presentadores sobre $\{a, b, c, d, x, y\}$ dan las ranuras compatibles $N(a) = \{0, 1\}$, $N(b) = \{1\}$, $N(c) = \{1, 2\}$, $N(d) = \{3, 4\}$, $N(x) = \{2\}$, $N(y) = \{4\}$. El conjunto $I = \{a, b, c, d\}$ es un transversal parcial ($a \mapsto 0$, $b \mapsto 1$, $c \mapsto 2$, $d \mapsto 3$). El plano sombreado es la vecindad saturada del bloque tight $R = \{a, b, c\}$: tres elementos, tres ranuras, nada de sobra. Es el muro que rechaza a x y deja pasar a y .

Léase sobre el ejemplo de la Figura 2: para $I = \{a, b, c, d\}$ todo subconjunto tiene al menos tantas ranuras como elementos, luego I es factible. Para `insert x I` el subconjunto $\{a, b, c, x\}$ tiene cuatro elementos y solo las tres ranuras $\{0, 1, 2\}$; Hall falla de inmediato. Lo que importa a continuación es que ese fallo tiene una *forma*. La pregunta que se pasa adelante es: cuando falla todo intento de inserción, ¿qué comparten esos fallos?

4. Por qué un transversal mayor debe ceder un elemento

Sean I, J transversales parciales con $|I| < |J|$, y supongamos por reducción al absurdo que ningún $e \in J \setminus I$ puede insertarse en I .

Para cada uno de esos e , la caracterización de Hall da un $S \subseteq \text{insert } e I$ con $|N(S)| < |S|$. Como el propio I cumple Hall, S tiene que contener a e (si no, S sería un subconjunto violador dentro de I); esto es `hall_violation_insert_mem`. Quitemos e , llamemos R al resto, y un breve estrujón de cardinales eleva la desigualdad estricta a cuatro hechos exactos:

$$R \subseteq I, \quad e \notin R, \quad |N(R)| = |R|, \quad N(e) \subseteq N(R).$$

El testigo tight (receta)

Llamemos a R **tight** cuando satura exactamente sus ranuras compatibles, $|N(R)| = |R|$ (`IsTight`): no le queda holgura ninguna. Un $R \subseteq I$ tight con $N(e) \subseteq N(R)$ es un *certificado de que e no tiene adónde ir*—toda ranura que e podría usar ya está comprometida por elementos de R , y R no tiene margen para recolocarse. En Lean la extracción es `exists_tight_witness_of_not_insert`: una inserción fallida nunca es un fracaso de búsqueda sin explicación; siempre devuelve este objeto.

Sobre el ejemplo corriente con $I = \{a, b, c, d\}$, el intento de añadir x devuelve el testigo tight $R = \{a, b, c\}$: tres elementos llenando las tres ranuras $\{0, 1, 2\}$, y $N(x) = \{2\}$ queda dentro de ellas (Figura 3).

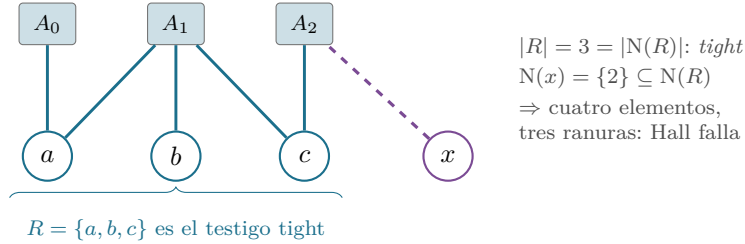


Figura 3: Una inserción fallida, explicada y no solo observada. Las tres ranuras sombreadas están saturadas por $\{a, b, c\}$, y toda ranura que x pudiera ocupar ya está entre ellas. El certificado es un conjunto, no el rastro de una búsqueda fallida.

Un conjunto tight es local. El aumento necesita una *única* obstrucción que cubra todo $J \setminus I$ a la vez, y el hecho decisivo es que la propiedad tight sobrevive a la unión. Si $R, S \subseteq I$ son tight entonces, como $N(R \cup S) = N(R) \cup N(S)$ y $N(R \cap S) \subseteq N(R) \cap N(S)$,

$$|N(R \cup S)| + |N(R \cap S)| \leq |N(R)| + |N(S)| = |R| + |S| = |R \cup S| + |R \cap S|,$$

mientras que Hall acota cada término de la izquierda por debajo con su pareja de la derecha. Ambas desigualdades quedan así forzadas a ser igualdades, y $R \cup S$ es tight (`tight_union`); la inducción da la versión para familias finitas, `tight_biUnion`.

Elijamos ahora un testigo tight R_e para cada $e \in J \setminus I$, pongamos $U = \bigcup_e R_e$ y apliquemos Hall a $P = (J \setminus I) \cup (U \cap J) \subseteq J$. Su vecindad queda dentro de $N(U)$, y U es tight, de modo que la aritmética de cardinales da $|J \setminus I| \leq |I \setminus J|$ (`card_sdiff_le_of_tight_cover`). Pero para conjuntos finitos $|I| < |J|$ equivale a $|I \setminus J| < |J \setminus I|$. Contradicción, y el aumento se sigue:

```

theorem partialTransversal_augmentation
  (hI : IsPartialTransversal A I) (hJ : IsPartialTransversal A J)
  (hcard : I.card < J.card) :
  ∃ e ∈ J \ I, IsPartialTransversal A (insert e I)

```

Esta es una prueba del aumento por Hall y conjuntos tight. No implementa emparejamiento máximo ni búsqueda de caminos de aumento: un teorema de existencia y un algoritmo certificado son entregables distintos, y aquí solo se reclama el primero.

5. Las tres capas de la prueba

La dificultad no es el conjunto vacío, ni la restricción, ni el teorema de Hall, que Mathlib ya proporciona. La verdadera ingeniería de pruebas es el trayecto que va de *varias* inserciones fallidas a *una* contradicción global. Escrito como un solo argumento es un párrafo; escrito como un solo bloque de tácticas es ilegible e irreutilizable. Por eso el desarrollo lo separa en tres capas, y esa separación es la afirmación técnica central del artículo.

Las tres capas

Capa de emparejamiento. `compatibleSlots`, `IsPartialTransversal`, la restricción y `partialTransversal_iff_hall`: una interfaz estable entre asignaciones inyectivas y desigualdades de cardinales. Todo lo que hay por encima habla solo de cardinales.

Capa de conjuntos tight. `exists_tight_witness_of_not_insert` convierte un fallo en un objeto; `tight_union` y `tight_biUnion` le dan a ese objeto un álgebra. Esta capa no menciona matroides ni emparejamientos—solo familias de conjuntos y la desigualdad de Hall—, y justo por eso es reutilizable, y por eso la Sección 7 cuesta casi nada.

Capa de conteo. `card_sdiff_le_of_tight_cover` toma una familia finita de subconjuntos tight que cubren las ranuras de $J \setminus I$ y devuelve la única desigualdad $|J \setminus I| \leq |I \setminus J|$. El teorema de aumento se limita entonces a elegir un testigo por cada elemento del subtipo finito $J \setminus I$ e invocarla.

Este es el análogo formal de dibujar la obstrucción antes de contarla. En Lean los detalles son concretos: los testigos se indexan por un subtipo finito y se eligen con `Exists.choose`; su unión es un `Finset.biUnion`; una vecindad es ella misma un `biUnion`; y la contradicción final es aritmética de unión, intersección y diferencia de `Finset` despachada por `omega`. Ninguno es profundo por separado. Mantenerlos en lemas con nombre propio es lo que impide que un párrafo combinatorio breve se convierta en una prueba opaca, y es lo que deja la capa de conjuntos tight disponible para quien la quiera con otro fin.

La afirmación merece decirse sin matices: *la novedad no es que Lean acepte la prueba; es que la arquitectura de la prueba aísla la obstrucción tight en lemas reutilizables*. Eso es a la vez una decisión matemática—método de obstrucción en lugar de caminos de aumento—y una decisión de ingeniería de pruebas, y la Sección 7 es la evidencia de que la decisión fue la correcta.

6. Empaquetar el matroide

`IndepMatroid.ofFinset` de Mathlib convierte un predicado de independencia sobre conjuntos finitos que cumpla los axiomas del vacío, hereditario y de aumento en un matroide finitario. El desarrollo lo aplica con conjunto base `Set.univ` y predicado `IsPartialTransversal A`:


```
noncomputable def transversalMatroid (A : Fin t → Finset α) : Matroid α
```

El conjunto base universal es deliberado. Un elemento de α que no aparezca en ningún conjunto presentador es sencillamente un *lazo*: ningún transversal parcial lo contiene. El teorema final es por tanto exacto sin restringir antes α a $\bigcup_i A_i$:

```
theorem transversalMatroid_indep (A : Fin t → Finset α) (I : Finset α) :
  (transversalMatroid A).Indep (I : Set α) ↔ IsPartialTransversal A I
```

Este es el teorema del matroide transversal para presentaciones finitas, como un objeto que los resultados de matroides de Mathlib—rango, clausura, bases, dualidad, menores—pueden consumir directamente; el Corolario 1 ejercita sobre él el operador de clausura, de modo que la afirmación queda demostrada y no solo enunciada. Se mantiene la distinción entre un teorema *disponible* y una aplicación *implementada*: la construcción pone la interfaz a disposición; no implementa aquí ningún optimizador voraz con pesos, ni un oráculo de rango, ni análisis de complejidad alguno.

7. La obstrucción es canónica

A la Sección 4 solo le hacía falta que los conjuntos tight fuesen cerrados por *unión*. Hágasele a la capa de conjuntos tight una pregunta más—¿y la intersección?—y el mismo estrujón submodular la responde, porque la cadena de desigualdades de arriba fuerza a la vez la igualdad en *ambos* lados. En Lean esto es un único lema que devuelve una conjunción, `tight_union_and_inter`; la mitad de la intersección, `tight_inter`, es la mitad que el aumento no usa jamás.

Proposición 1 (Los conjuntos tight forman un retículo; `tight_union_and_inter`). *Sea I un transversal parcial y sean $R, S \subseteq I$ tight. Entonces $R \cup S$ y $R \cap S$ son tight. Por tanto los subconjuntos tight de I forman un subretículo del retículo booleano sobre I , que contiene a $\emptyset$.*

Que los casos de igualdad de la condición de Hall sean cerrados por unión e intersección es teoría clásica de emparejamientos—es la submodularidad de N , el motor tras la fórmula de defecto de Ore [13] y la descomposición de Dulmage–Mendelsohn [14], y es estándar en Lovász y Plummer [15]. No se reclama nada nuevo en la matemática. Lo nuevo es que el desarrollo por capas lo consigue al precio de añadir un miembro más a la conjunción, y que la consecuencia de abajo puede entonces siquiera enunciarse.

Como la familia es finita y cerrada por unión, tiene máximo. El desarrollo define

$$\text{tightSubsets } A I = \{R \in \mathcal{P}(I) : R \text{ tight}\}, \quad \text{maxTight } A I = \bigcup \text{tightSubsets } A I,$$

y prueba que esa unión es ella misma tight (`maxTight_isTight`, vía `tight_biUnion`) y contiene a todo subconjunto tight (`subset_maxTight`). Es el *máximo*, no meramente un elemento maximal.

La recompensa es un enunciado de decisión. El lema de existencia de la Sección 4 dice que una inserción fallida *produce* un recubrimiento tight; su recíproco es elemental—si $R \subseteq I$ es tight, $e \notin R$ y $N(e) \subseteq N(R)$, entonces `insert e R` tiene $|R| + 1$ elementos y solo $|R|$ ranuras, luego Hall falla (`not_insert_of_tight_cover`). Necesario y suficiente a la vez; y entonces la monotonía de N colapsa el existencial sobre el máximo:

Teorema 1 (Un solo conjunto decide toda inserción; `insert_iff_not_subset_maxTight`). *Sea I un transversal parcial de A y sea $e \notin I$. Entonces*

$$\text{insert } e I \text{ es un transversal parcial} \iff N(e) \not\subseteq N(\text{maxTight } A I).$$

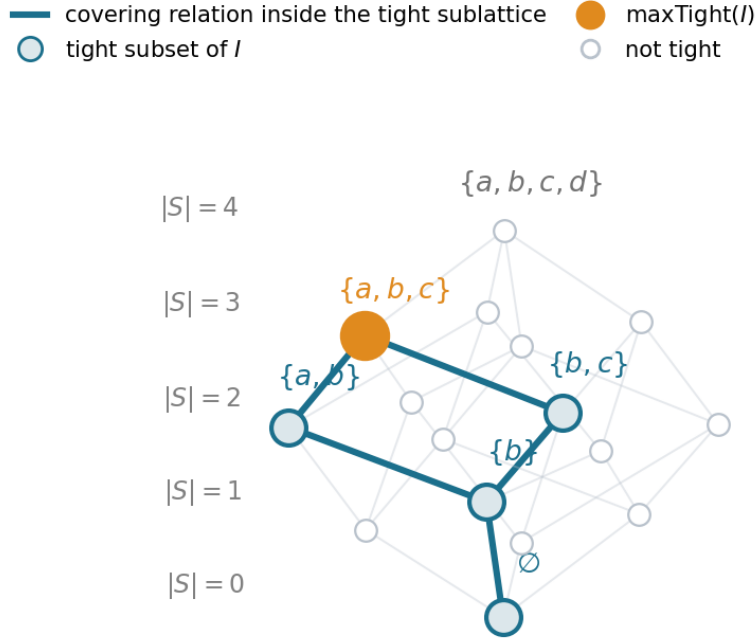


Figura 4: El retículo de subconjuntos tight de $I = \{a, b, c, d\}$, dibujado dentro del retículo booleano sobre I (la altura es $|S|$); $\text{maxTight}(I)$ lo corona, y el propio I no es tight. Solo cinco de los dieciséis subconjuntos son tight, y forman un retículo de verdad, no una cadena: $\{a, b\} \cap \{b, c\} = \{b\}$ y $\{a, b\} \cup \{b, c\} = \{a, b, c\}$ son ambos tight (Proposición 1). La cima, en ámbar, es $\text{maxTight}(I) = \{a, b, c\}$ —un subconjunto *propio* de I , ya que I tiene cuatro elementos y cinco ranuras. Una sola prueba de contención contra su vecindad decide toda inserción.

El criterio queda enunciado arriba en el vocabulario propio del desarrollo. Puede decirse en el de Mathlib, y esa es la forma en que se convierte en una afirmación sobre un matroide y no sobre un predicado hecho a medida:

Corolario 1 (La obstrucción calcula la clausura; `mem_closure_iff_subset_maxTight`). *Con I y e como arriba,*

$$e \in (\text{transversalMatroid } A).\text{closure } I \iff N(e) \subseteq N(\text{maxTight } A I).$$

`Matroid.closure` es el operador propio de Mathlib, y arrastra consigo su teoría de planos, conjuntos generadores y rango; el corolario dice que, sobre conjuntos independientes, ese operador queda decidido por una sola prueba de contención contra un único conjunto finito canónico—sin búsqueda sobre subconjuntos y sin recalculr ningún emparejamiento. La prueba son tres líneas: el `Indep.notMem_closure_iff_of_notMem` de Mathlib convierte la pertenencia a la clausura en el fallo de una inserción, `transversalMatroid_indep` convierte eso en una afirmación sobre transversales parciales, y el Teorema 1 remata. Con esto queda además saldada la promesa de la Sección 6: el aparato de la biblioteca sí se aplica al objeto construido, y aquí se le está aplicando.

Esta es la respuesta a la reserva que deja abierta *Theorem of the Day*—¿cómo sabemos que nuestros transversales parciales son válidos sin encontrar un emparejamiento al mismo tiempo? Para un I fijo no hace falta: un solo conjunto tight, calculado una vez, responde toda pregunta posterior de pertenencia mediante una prueba de contención. En el ejemplo corriente $\text{maxTight} = \{a, b, c\}$ con $N = \{0, 1, 2\}$, de modo que $N(x) = \{2\}$ queda dentro y x es rechazado, mientras que $N(y) = \{4\}$

queda fuera e y es admitido (Figura 4). Un contraste exhaustivo sobre esa presentación—las 56 transversales parciales y las 180 decisiones de inserción—coincide con el teorema en todos los casos.

Dos matices honestos. Primero, `maxTight` se define mediante un filtro sobre el conjunto de partes de I , así que la *definición* es exponencial; el teorema certifica la caracterización, no un procedimiento eficiente, y calcular el máximo conjunto tight de forma eficiente (es una cuestión de Dulmage–Mendelsohn) se deja a la literatura algorítmica. Segundo, igual que en la Proposición 1, la matemática es clásica; la contribución es que sea formal, y que haya caído de la arquitectura en lugar de exigir un desarrollo nuevo.

all 56 partial transversals of the presentation; 180 insertion decisions,
every one of them predicted by $N(e) \subseteq N(\text{maxTight}(I))$

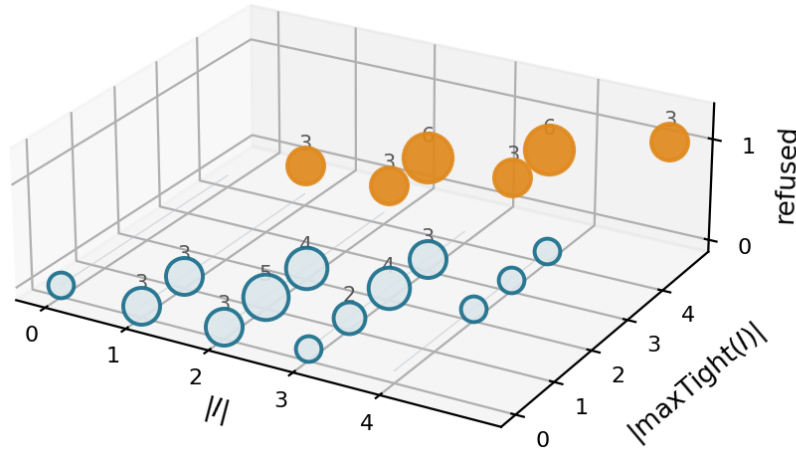


Figura 5: El criterio, ejercitado sobre la presentación entera y no sobre un solo ejemplo. Cada transversal parcial I de la familia corriente se sitúa en $(|I|, |\text{maxTight}(I)|, \#\text{rechazados})$, donde un rechazo significa un elemento fuera de I que no puede insertarse; el área del marcador cuenta cuántos I comparten posición. El ámbar marca las posiciones donde la obstrucción canónica muere de verdad. Las 56 transversales parciales y las 180 decisiones de inserción concuerdan con el Teorema 1; la gráfica es la comprobación exhaustiva de `check_canonical_obstruction.sage`, no una muestra.

8. Las piezas

La Tabla 1 reúne los resultados que sostienen el edificio; la Tabla 2 mide el desarrollo. La cadena de dependencias es corta: la capa de emparejamiento da la equivalencia de Hall; la equivalencia de Hall da el testigo tight; `tight_union_and_inter` dota a los conjuntos tight de un álgebra; la capa de conteo convierte un recubrimiento en una sola desigualdad; y el retículo da la obstrucción canónica.

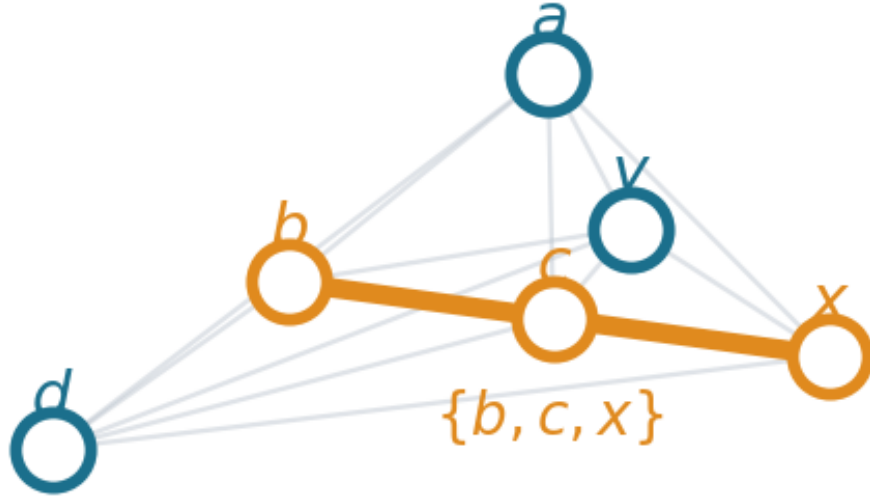


Figura 6: La misma presentación vista como el matroide que define: rango 5 sobre seis elementos, luego exactamente un circuito. Doce de los trece planos de rango 2 llevan dos puntos; uno lleva tres, y el dibujo lo respeta colocando b , c y x sobre una recta. Esa recta *es* el violador de Hall, $N(\{b, c, x\}) = \{A_1, A_2\}$ —tres puntos compitiendo por dos ranuras—y todo rechazo en cualquier punto de esta presentación se remonta a ella. El circuito y la obstrucción tight son el mismo fenómeno leído desde lados opuestos: un conjunto tight es lo que queda de un circuito al quitarle uno de sus elementos.

Nombre Lean	Enunciado	Estado
<i>TransversalMatroid.lean</i> — la construcción		
<code>partialTransversal_empty</code>	$\emptyset$ es un transversal parcial	probado
<code>partialTransversal_subset</code>	restricción de un emparejamiento (herencia)	probado
<code>partialTransversal_iff_hall</code>	factible $\iff S \leq N(S) $ en todo $S \subseteq I$	probado
<code>hall_violation_insert_mem</code>	un subconjunto violador contiene a e	probado
<code>exists_tight_witness_of_not_insert</code>	inserción fallida $\Rightarrow$ testigo tight	probado
<code>tight_union, tight_biUnion</code>	lo tight sobrevive a la unión finita	probado
<code>card_sdiff_le_of_tight_cover</code>	$ J \setminus I \leq I \setminus J $ desde un recubrimiento	probado
<code>partialTransversal_augmentation</code>	el axioma de intercambio	probado
<code>transversalMatroid_indep</code>	$\text{Indep} \iff$ transversal parcial	probado
<i>TransversalObstruction.lean</i> — la obstrucción canónica		
<code>tight_union_and_inter</code>	los conjuntos tight forman un retículo	probado
<code>maxTight_isTight</code>	la cima del retículo es tight	probado
<code>subset_maxTight</code>	es el máximo, no solo maximal	probado
<code>not_insert_of_tight_cover</code>	un recubrimiento tight bloquea la inserción	probado
<code>not_insert_iff_exists_tight</code>	fallo $\iff$ algún recubrimiento tight	probado
<code>insert_iff_not_subset_maxTight</code>	un conjunto decide toda inserción	probado
<code>mem_closure_iff_subset_maxTight</code>	calcula <code>Matroid.closure</code>	probado

Cuadro 1: El desarrollo, entero sin `sorry`, en Lean 4 / Mathlib. `#print axioms` sobre `partialTransversal_augmentation`, `transversalMatroid_indep`, `tight_union_and_inter`, `maxTight_isTight`, `insert_iff_not_subset_maxTight` y `mem_closure_iff_subset_maxTight` devuelve únicamente `propext`, `Classical.choice` y `Quot.sound`.

Magnitud	Valor
Líneas: construcción / obstrucción	400 / 245
Teoremas + definiciones	15+4 / 19+5
Importaciones directas de Mathlib	3: <code>Combinatorics.Hall.Basic</code> , <code>Matroid.IndepAxioms</code> , <code>Matroid.Closure</code>
<code>sorry / admit / native_decide</code>	0
Axiomas más allá del núcleo de Lean	0: solo <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
Compilación	<code>lake build Theorems.TransversalObstruction</code> — 1164 tareas, salida 0
Compilación, Mathlib en caliente	≈ 4.7 s (módulo de la obstrucción)
Cadena de herramientas	Lean 4 v4.32.0; Mathlib 84fdd0bf (2026-07-15)
Contraste Sage	56 transversales parciales, 180 decisiones de inserción

Cuadro 2: El desarrollo, medido. La huella es deliberadamente ligera: dos importaciones de Mathlib y nada de álgebra pesada, porque el argumento es conteo elemental una vez separadas las capas. La fila de Sage [28] es un contraste ejecutable del ejemplo corriente, no evidencia de los teoremas; esa evidencia es el núcleo de Lean.

9. El límite de la contribución

Qué está probado. El teorema del matroide transversal para presentaciones finitas $A : \text{Fin } t \rightarrow \text{Finset } \alpha$: los transversales parciales cumplen los tres axiomas de conjunto independiente y son exactamente los conjuntos finitos independientes de `transversalMatroid A`; más el retículo de conjuntos tight, el subconjunto tight máximo y el criterio de inserción del Teorema 1. Todo sin `sorry`, con el perfil de axiomas de arriba.

Solo presentaciones finitas. La familia es finita y cada A_i es finito. El tipo ambiente α no tiene por qué ser finito, pero el desarrollo caracteriza los conjuntos independientes *finitos* y usa la construcción finitaria de Mathlib. La versión infinita—la que Mirsky y Perfect probaron también [3]—no queda cubierta.

Ninguna afirmación algorítmica. La prueba selecciona testigos tight con elección clásica. Prueba el aumento; no implementa búsqueda de caminos de aumento, ni emparejamiento máximo, ni cota alguna. `maxTight` se define por un filtro sobre el conjunto de partes y no es un procedimiento eficiente.

La matemática es clásica. Edmonds–Fulkerson y Mirsky–Perfect probaron el teorema; el método de conjuntos tight es teoría estándar de transversales y emparejamientos [13, 14, 15, 16, 17]. Lo que se ofrece aquí es la formalización, su arquitectura, y el empaquetado de la obstrucción como objeto canónico.

Autoría. La formalización se llevó a cabo en interacción estrecha con un asistente de IA (Claude, Anthropic): la elección del objetivo, la arquitectura de la prueba, los enunciados y las afirmaciones de alcance y novedad son míos; el asistente redactó e iteró buena parte del Lean bajo esa dirección. Esto no afecta a la validez—una prueba se acepta solo cuando el núcleo de Lean la verifica, cosa que hace, con el perfil de axiomas indicado arriba—, pero se declara sin rodeos por atribución.

Frontera de confianza. Los tres axiomas indicados describen los teoremas tal como los verifica el entorno de Lean actual. No son una afirmación de haber eliminado el núcleo de Lean, ni el compilador, ni el hardware, del círculo de confianza.

10. Prioridad: qué se buscó, cuándo, y qué la refutaría

Las afirmaciones del tipo “esto no estaba formalizado” se degradan. Son ciertas de un momento, de un conjunto de bibliotecas y de un conjunto de consultas, y las tres cosas cambian. Esta sección fija las tres para que quien lea dentro de unos años pueda *comprobar* la afirmación en lugar de confiar en ella y, si ha dejado de ser cierta, pueda decir exactamente cuándo dejó de serlo.

Qué se afirma. Dos cosas, y nada más allá de ellas. (i) Hasta donde sé, el matroide `transversalMatroid` es el primero de Mathlib construido a partir de una presentación combinatoria y no por una construcción trivial o dual. (ii) Hasta donde sé, el teorema del matroide transversal no había sido formalizado en ningún asistente de pruebas antes de este desarrollo.

Ambas son afirmaciones sobre la *formalización*, nunca sobre la matemática, que es de Edmonds–Fulkerson y de Mirsky–Perfect y tiene sesenta años.

Qué no se afirma. Ninguna prioridad sobre enunciado matemático alguno de este artículo. Ninguna afirmación de que el área circundante esté sin formalizar—no lo está: el AFP contiene algoritmos verificados para matroides y greedoides, incluido el emparejamiento bipartito máximo [23], y Mathlib contiene una teoría estructural de matroides sustancial. Y ninguna afirmación sobre trabajo que no pude leer, incluidos desarrollos inéditos, repositorios privados y formalizaciones archivadas bajo otro nombre.

Qué se buscó exactamente, y cuándo. Todo lo siguiente, en julio de 2026.

- **Mathlib**, revisión 84fdd0bf (15-07-2026), cadena de herramientas v4.32.0: una búsqueda de texto sin distinción de mayúsculas de *transversal* sobre todo Mathlib/, que devuelve `Combinatorics/Hall/Basic.lean` (solo en prosa) y nueve ficheros de teoría de grupos (`GroupTheory/Complement`, `CosetCover`, `Schreier`, `SchurZassenhaus`, `Transfer` y otros), y ninguna aparición de *transversal matroid*; más una lectura de todas las declaraciones de `Combinatorics/Matroid/Constructions.lean`, que define exactamente `emptyOn`, `loopyOn`, `freeOn` y `uniqueBaseOn`.
- **Isabelle/AFP**: el índice temático de combinatoria, y búsquedas de *transversal matroid*, *transversal*, *matroid* y *marriage*. Encontrado: *Matroids* (Keinholz, 2018) [21], *Hall’s Marriage Theorem* (Jiang y Nipkow, 2010) [20], y el trabajo de ITP 2025 sobre algoritmos de matroides y greedoides [23]. Ninguna entrada de matroide transversal localizada.
- **Mizar, Coq/mathcomp, HOL Light**: búsquedas con los mismos términos. Se localizaron teoremas vecinos sobre conjuntos finitos en Coq [22]; no se localizó ninguna formalización del matroide transversal en ninguno de los tres. Esta es la pata más débil de la auditoría: se apoya en búsqueda pública y no en la lectura del índice de cada biblioteca, y consideraría que un aviso por aquí tiene más probabilidad de acertar que uno dirigido a Mathlib o al AFP.

Qué la refutaría. Un único localizador—biblioteca, nombre de la entrada, fichero, fecha—de una formalización del teorema del matroide transversal, o de cualquier matroide de Mathlib construido a partir de datos combinatorios anterior a la revisión 84fdd0bf, refuta de plano la afirmación correspondiente. Nada más en este artículo depende de ninguna de las dos: el desarrollo en Lean, el perfil de axiomas, el retículo de conjuntos tight y el Teorema 1 se sostienen o caen por el núcleo, no por la prioridad. Agradecería que se me enviara ese localizador, y lo dejaría consignado.

11. Conclusión

He formalizado el teorema del matroide transversal para presentaciones finitas en Lean 4 sobre Mathlib, y lo he empaquetado como objeto matroide, `transversalMatroid A`, cuyos conjuntos

finitos independientes identifica exactamente `transversalMatroid_indep`. Hasta donde sé—frente a las búsquedas fechadas y detalladas en la Sección 10, y con los límites que allí se declaran—es el primer matroide de Mathlib construido a partir de datos combinatorios y no de una construcción trivial o dual.

He probado el aumento—el único axioma que la definición no regala—por el método de obstrucción de Hall y no por caminos de aumento, y he organizado esa prueba en tres capas independientes: una interfaz de emparejamiento sobre el teorema de Hall finito de Mathlib, una capa de conjuntos tight con álgebra propia, y una capa de conteo que convierte un recubrimiento de conjuntos tight en una única contradicción de cardinales. Cada paso es un lema con nombre propio y reutilizable por separado.

El objeto resultante se integra directamente con Mathlib. Se produce mediante `IndepMatroid.offFinset`, de modo que la teoría de rango, clausura, bases, dualidad y menores de la biblioteca se le aplica sin trabajo adicional, y el conjunto base se toma universal, de manera que los elementos ajenos a todo conjunto presentador son lazos y la caracterización es exacta. Esa integración se ejerce, no se declara: el Corolario 1 identifica la propia `Matroid.closure` de Mathlib, sobre conjuntos independientes, con una sola prueba de contención contra `maxTight`.

La arquitectura se pagó sola. Los conjuntos tight son cerrados por intersección además de por unión, luego forman un retículo; todo transversal parcial tiene un subconjunto tight máximo; y una sola prueba de contención contra su vecindad decide toda inserción (Teorema 1). La obstrucción no es un incidente de una búsqueda fallida, sino un invariante canónico del par (presentación, conjunto independiente).

Trabajo futuro, en el orden en que lo intentaría: extraer un procedimiento ejecutable que devuelva o bien un aumento o bien el certificado tight, y conectarlo con los algoritmos de emparejamiento verificados de la literatura de Isabelle [23]; llevar el Corolario 1 hasta el rango, donde el valor debería ser la fórmula de defecto de Ore $r(X) = \min_{S \subseteq X} (|X \setminus S| + |N(S)|)$ [13]—verificada exhaustivamente sobre la presentación corriente por `check_canonical_obstruction.sage` y no formalizada, que es justo por lo que es trabajo futuro y no un resultado; elevar la construcción a familias indexadas infinitas, siguiendo a Mirsky y Perfect y la propia vía de compacidad de Mathlib para Hall; y promover la capa de conjuntos tight a contribución autónoma de Mathlib, ya que trata de sistemas finitos de conjuntos y no menciona matroide alguno.

12. Reflexiones

Formalizar un teorema de mediados de los sesenta no es, en sí, interesante. Lo que enseñó el ejercicio sí lo es, y estas siete observaciones son el residuo.

1. Un teorema no es el mismo objeto que la razón por la que es cierto. El teorema del matroide transversal afirma que algo *siempre puede* hacerse: un transversal mayor siempre cede un elemento. La prueba, en cambio, procede enteramente estudiando lo que *no puede* hacerse. Formalizar sacó esa asimetría a la luz, porque un núcleo no acepta “se ve que algún elemento tiene que servir”; exige una construcción, y la única construcción disponible es la del obstáculo. El objeto negativo resultó ser el más rico—lleva adjuntos un retículo, un máximo y un procedimiento de decisión—, mientras que el enunciado positivo no tiene nada de eso. Merece enunciarse como heurística de trabajo: cuando un teorema afirma que cierta elección existe, el contenido formalizable suele vivir en la clasificación de las razones por las que podría no existir.

2. Las bibliotecas formales son ricas en teoría y pobres en ejemplos. Mathlib sabe calcular el dual, los menores, el rango y la clausura de un matroide, y en la revisión 84fdd0bf no tenía prácticamente ningún matroide al que aplicárselo. El desequilibrio es estructural y no accidental: la teoría se enuncia una vez y se reutiliza, mientras que cada familia de ejemplos—gráficos, lineales, transversales—cuesta su propia construcción, y las construcciones se hacen las últimas. El mismo patrón se ve en cualquier biblioteca grande. Si uno quiere ser útil a una biblioteca formal, y no solo estar presente en ella, construir un ejemplo que su teoría existente pueda masticar puede valer más que otro teorema sobre objetos que nadie ha instanciado.

3. Las capas no fueron pulcritud: fueron adonde se fue la matemática. Partir la prueba en una capa de emparejamiento, una de conjuntos tight y una de conteo se hizo por legibilidad. Resultó ser el acto sustantivo. Una vez que los conjuntos tight tuvieron capa propia, preguntar si eran cerrados por intersección era una pregunta natural cuya respuesta costaba un miembro más en la conjunción, y el Teorema 1 vino detrás. Si la prueba del aumento se hubiera escrito como un solo argumento—como está en los libros de texto, y como habría salido en un primer intento en Lean—, el retículo habría quedado invisible, porque no habría habido objeto al que hacerle la pregunta. Ingeniería de pruebas y descubrimiento matemático fueron, aquí, la misma actividad.

4. Un certificado vale más que una búsqueda. Un algoritmo de caminos de aumento responde a “¿puede insertarse e ?” ejecutándose, y vuelve a responder ejecutándose otra vez. El conjunto tight responde una sola vez y queda reutilizable para toda pregunta posterior sobre el mismo I . La reserva que la propia página de *Theorem of the Day* pone a este teorema—que la selección voraz “deja abierta la cuestión de cómo sabemos que nuestros transversales parciales son válidos si no encontramos un emparejamiento al mismo tiempo”—es precisamente una queja sobre la búsqueda, y la obstrucción canónica es la respuesta. Que un rechazo venga con su razón adjunta es un principio de diseño que generaliza mucho más allá de los matroides.

5. Clásico no quiere decir disponible. Todos los ingredientes matemáticos de aquí tienen décadas: Hall en 1935, el teorema mismo en 1965–67, y la submodularidad de la función de vecindad es más antigua que ambos. Y sin embargo ningún asistente de pruebas tenía el enunciado, y la razón no es la dificultad. Es que los resultados clásicos se ensamblan a partir de prerequisites que las bibliotecas adquieren en un orden distinto al de los libros de texto, de modo que el coste marginal de un teorema dentro de una biblioteca tiene poco que ver con su coste marginal sobre el papel. Esto corta por los dos lados: explica por qué algunos teoremas célebres siguen sin formalizar, y significa que un resultado clásico modesto puede ser aun así una contribución real a una biblioteca.

6. La definición correcta cuesta menos que la prueba ingeniosa. Definir `IsPartialTransversal` como la no vacuidad del `hallMatchingsOn` de Mathlib, y no como un predicado a medida, fue la única decisión que hizo corto el desarrollo. Compró de golpe la dirección recíproca de Hall, y convirtió todo lema posterior en una afirmación sobre cardinales en lugar de sobre funciones. La tentación contraria—definir un predicado cómodo y luego probar que equivale al de la biblioteca—habría añadido una capa de traducción a cada prueba. Cuando una biblioteca ya habla de tu objeto, casi siempre sale más barato hablar su dialecto que introducir el propio.

7. Qué cambió el asistente, y qué no. Este desarrollo se escribió en interacción estrecha con un asistente de IA, y conviene ser preciso sobre el efecto. Comprimió la distancia mecánica entre una idea y una prueba verificada, de modo que en una tarde cupieron más variantes arquitectónicas de

las que habrían cabido en una semana—y fue esa iteración barata, y no ninguna intuición concreta, la que sacó a la superficie la mitad de la intersección de la Proposición 1. Lo que no cambió es el criterio de verdad. El núcleo aceptó o rechazó, el informe de axiomas lo genera la máquina, y las declaraciones de alcance de la Sección 9 son afirmaciones de las que responde una persona. Una forma más rápida de probar cosas no baja el listón; si acaso, hace más importante enunciarlo de forma explícita.

13. Cabos sueltos

Cuatro preguntas que el trabajo plantea y no resuelve.

1. *Del testigo al algoritmo.* El testigo tight ya explica un rechazo. ¿Puede convertirse la prueba de existencia en un procedimiento verificado que devuelva o bien un transversal mayor o bien un certificado comprobable de vecindad saturada—y hace el máximo conjunto tight, obtenido de una descomposición de Dulmage–Mendelsohn, que ese procedimiento sea polinómico con el certificado de regalo?
2. *Qué finitud hace falta de verdad.* La prueba usa la finitud en tres sitios distintos: para elegir testigos, para tomar una unión finita y para hacer aritmética de cardinales. ¿Cuál de los tres sobrevive a presentaciones infinitas, y basta el argumento de compacidad de Mathlib para Hall para cargar con el resto?
3. *¿Pertenece a Mathlib la capa de conjuntos tight?* `tight_union_and_inter` y `tight_biUnion` no dicen nada sobre matroides. Son lemas sobre un sistema finito de conjuntos y el caso de igualdad de la condición de Hall. Eso es un enunciado con forma de Mathlib buscando casa.
4. *Una obstrucción, tres caras.* El retículo de conjuntos tight es el mismo fenómeno que la descomposición de Dulmage–Mendelsohn de un grafo bipartito y que el violador maximal canónico en optimización submodular. ¿Existe una única noción en Lean—una función submodular y su retículo de igualdad—que subsuma las tres, con el caso transversal como instancia? Esa es la cuenta del collar que más me gustaría seguir enhebrando.

Reproducir

```
lake build Theorems.TransversalMatroid      # la construccion
lake build Theorems.TransversalObstruction  # la obstruccion canonica
lake env lean formalizations/transversal-matroid/lean/
        Theorems/AxiomReport.lean          # el informe de confianza
python formalizations/transversal-matroid/sage/check_canonical_obstruction.sage
```

Lean 4, cadena de herramientas `leanprover/lean4:v4.32.0`, sobre Mathlib en la revisión `84fdd0bf` (2026-07-15):

<https://github.com/leanprover-community/mathlib4>
fijada en `84fdd0bfb8e85237f54b9617cca79c6198d40a75`

Los dos módulos de Mathlib sobre los que se apoya este desarrollo, en esa revisión, son `Combinatorics/Hall/Basic.lean` and `Combinatorics/Matroid/IndepAxioms.lean`; y el módulo de construcciones cuyo contenido reporta la Sección 10 es

`Combinatorics/Matroid/Constructions.lean`. Toda afirmación de ausencia de este artículo es comprobable contra esas URL exactas, y lo sigue siendo después de que Mathlib avance. Para reproducirlo en otra máquina, apúntese `lakefile.lean` a una Mathlib compilada con la misma cadena de herramientas; la ruta local usada aquí está registrada en `MATHLIB_ENVIRONMENT.md`.

Las fuentes en Lean, el informe de axiomas generado, el contraste exhaustivo en SageMath y el generador determinista de figuras están archivados junto con las demás formalizaciones de esta serie [29, 30, 31] en <https://github.com/karlesmarin/godsil-gutman-lean>. Ambos ficheros Lean están sin `sorry`, y `#print axioms` sobre los seis teoremas listados en la Tabla 1 devuelve únicamente `propext`, `Classical.choice` y `Quot.sound`. El registro de verificación a nivel de fuente es `VERIFICATION.md`; y el ejemplo corriente de cada figura se rederiva de forma exhaustiva—las 56 transversales parciales y las 180 decisiones de inserción—mediante `check_canonical_obstruction.sage` antes de dibujar figura alguna.

Referencias

- [1] P. Hall, On representatives of subsets, *J. London Math. Soc.* **10** (1935), 26–30. <https://doi.org/10.1112/jlms/s1-10.37.26>.
- [2] J. Edmonds, D. R. Fulkerson, Transversals and matroid partition, *J. Res. Nat. Bur. Standards Sect. B* **69B** (1965), 147–153.
- [3] L. Mirsky, H. Perfect, Applications of the notion of independence to problems of combinatorial analysis, *J. Combin. Theory* **2** (1967), 327–357.
- [4] H. Perfect, Applications of Menger’s graph theorem, *J. Math. Anal. Appl.* **22** (1968), 96–111. [https://doi.org/10.1016/0022-247X\(68\)90163-7](https://doi.org/10.1016/0022-247X(68)90163-7).
- [5] R. Whitty, The Transversal Matroid Theorem, *Theorem of the Day*, <https://www.theoremoftheday.org/CombinatorialTheory/Transversal/TotDTransversal.pdf> (attribution: “proved in the mid-1960s by Jack Edmonds and Delbert Ray Fulkerson in the USA and, independently (and also in an infinite version) by Leon Mirsky and Hazel Perfect at Sheffield University in the UK”).
- [6] R. Rado, A theorem on independence relations, *Quart. J. Math. Oxford Ser.* **13** (1942), 83–89 (el criterio de Hall generalizado a matroides).
- [7] H. Perfect, *Studies in Transversal Theory with Particular Reference to Independence Structures and Graphs*, tesis doctoral, University of London, 1969.
- [8] H. Perfect, L. Mirsky, Spectral properties of doubly-stochastic matrices, *Monatsh. Math.* **69** (1965), 35–57. <https://doi.org/10.1007/BF01313442> (el origen de la conjetura de Perfect–Mirsky).
- [9] J. Oxley, The contributions of Dominic Welsh to matroid theory, <https://www.math.lsu.edu/~oxley/dominic.pdf> (sobre las escuelas de Oxford y Sheffield, y sobre el congreso de Edmonds de 1965 en el National Bureau of Standards).
- [10] H. Whitney, On the abstract properties of linear dependence, *Amer. J. Math.* **57** (1935), 509–533 (el artículo que da nombre al matroide, publicado el mismo año que el de Hall).
- [11] G. Farr, J. Oxley, The contributions of W. T. Tutte to matroid theory, https://www.math.lsu.edu/~oxley/Tutte_farrox5.pdf (sobre el letargo de la teoría de matroides tras 1935 y su renacer con los artículos de Tutte de 1958–59).
- [12] V. Bryant, H. Perfect, *Independence Theory in Combinatorics: An Introductory Account with Applications to Graphs and Transversals*, Chapman and Hall, 1980.

- [13] O. Ore, Graphs and matching theorems, *Duke Math. J.* **22** (1955), 625–639 (la forma de defecto del teorema de Hall).
- [14] A. L. Dulmage, N. S. Mendelsohn, Coverings of bipartite graphs, *Canad. J. Math.* **10** (1958), 517–534.
- [15] L. Lovász, M. D. Plummer, *Matching Theory*, North-Holland, 1986.
- [16] D. J. A. Welsh, *Matroid Theory*, Academic Press, 1976.
- [17] J. G. Oxley, *Matroid Theory*, 2nd ed., Oxford University Press, 2011.
- [18] A. W. Ingleton, Transversal matroids and related structures, in *Higher Combinatorics*, NATO ASI Series C **31**, Reidel, 1977, 117–131.
- [19] A. Gusakov, B. Mehta, K. A. Miller, Formalizing Hall’s marriage theorem in Lean, 2021. <https://arxiv.org/abs/2101.00127> (el desarrollo de Mathlib que se usa aquí: `Mathlib.Combinatorics.Hall.Basic`).
- [20] D. Jiang, T. Nipkow, Hall’s Marriage Theorem, *Archive of Formal Proofs* (2010). <https://isa-afp.org/entries/Marriage.html>.
- [21] J. Keinholtz, Matroids, *Archive of Formal Proofs* (2018). <https://isa-afp.org/entries/Matroids.html>.
- [22] A. Singh, R. Natarajan, Fully mechanized proofs of Dilworth’s theorem and Mirsky’s theorem, 2017. <https://arxiv.org/abs/1703.06133> (Coq; teoremas vecinos sobre conjuntos finitos).
- [23] *A Formal Analysis of Algorithms for Matroids and Greedoids*, *Interactive Theorem Proving (ITP 2025)*, LIPIcs, Schloss Dagstuhl. <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.2> (incluye emparejamiento bipartito máximo ejecutable).
- [24] *Formalizing the Excluded Minor Characterization of Binary Matroids in the Lean Theorem Prover*, University of Waterloo, 2023. <https://uwspace.uwaterloo.ca/items/d35d17be-485d-4f60-8696-4dce7ae907bb>.
- [25] *Composition Direction of Seymour’s Theorem for Regular Matroids — Formally Verified*, 2025. <https://arxiv.org/abs/2509.20539>.
- [26] The mathlib Community, The Lean mathematical library, in *Certified Programs and Proofs (CPP 2020)*, ACM, 2020, 367–381. <https://doi.org/10.1145/3372885.3373824>. Repository: <https://github.com/leanprover-community/mathlib4>; este trabajo se verifica contra la revisión 84fdd0bfb8e85237f54b9617cca79c6198d40a75 (2026-07-15).
- [27] L. de Moura, S. Ullrich, The Lean 4 theorem prover and programming language, in *Automated Deduction (CADE 28)*, LNCS **12699**, Springer, 2021, 625–635. https://doi.org/10.1007/978-3-030-79876-5_37. Repository: <https://github.com/leanprover-community/lean4>.
- [28] The Sage Developers, *SageMath, the Sage Mathematics Software System*. <https://www.sagemath.org>.
- [29] C. Marín, *When Separation Fails, a Canon Appears: one counting identity behind separation, synchronization and symmetry-breaking in permutation groups, formalized in Lean 4*, Zenodo, 2026. doi:10.5281/zenodo.21419285 (la cuenta anterior del collar tomada de *Theorem of the Day*; mismo método de selección, otro racimo).
- [30] C. Marín, *Random Signs into Matchings: A Godsil–Gutman Identity, Formalized in Lean 4*, Zenodo, 2026. doi:10.5281/zenodo.20517350 (emparejamientos de un grafo bipartito, contados en vez de elegidos).
- [31] C. Marín, *The Inward Bow of a Real-Rooted Polynomial: Newton’s Inequalities, Machine-Checked in Lean 4*, Zenodo, 2026. doi:10.5281/zenodo.20693064 (log-concavidad, el ancestro clásico de la log-concavidad de matroides).